

Manual de Arquitetura e Integração: Dashboard (Master)

Projeto Wearion- ResQSense

Data - 16/02/2026

Autor- Dinis

1. Visão Geral do Sistema (Lado do Tablet)

Este relatório documenta a infraestrutura de software desenvolvida para o Tablet.

Na arquitetura de rede do ResQSense, o Tablet assume o papel de Master. O seu principal objetivo não é apenas apresentar dados, mas também gerir ativamente a rede LoRa: registar os dispositivos na rede, atribuir janelas de tempo de transmissão aos coletes (para isso usamos o protocolo TDMA) para evitar colisões, e garantir a descriptação em tempo real do dados recebidos.

2. Ferramentas e Bibliotecas usadas (Python)

Para o Raspberry Pi conseguir comunicar com a rede de sensores, é necessário fazer uma transição da lógica do C++ para um ambiente Python. Para a implementação deste módulo em Python, vão ser usadas as seguintes bibliotecas:

- *adafruit-circuitpython-rfm9x*: Esta biblioteca faz a ligação SPI entre o sistema Linux e o rádio LoRa SX1276. Além disso, a potência de emissão vai ser limitada aos 14 dBm diretamente no código, para garantirmos que respeitamos as regras europeias (ETSI EN 300 220) para a banda dos 868 MHz.
- *pycryptodome*: Assegura a leitura dos pacotes encriptados nativamente pela biblioteca *mbedtls* do lado do hardware.

- *struct*: É uma biblioteca nativa, usada para empacotar e desempacotar os bytes de informação. Usamos isto para que o código imite a forma como o microcontrolador (em C++) organiza a memória.

3. Camada de Segurança e Sincronização (TDMA)

Para que o Python do tablet e o hardware do colete consigam entender-se e mantenham os dados seguros, implementámos o algoritmo AES-128.

Escolhemos o modo ECB (Electronic Codebook) porque este modo tranforma 16 bytes limpos noutros 16 bytes limpos, sem adicionar vetores de inicialização extras que iam estragar o tamanho limite do nosso pacote. A chave privada da equipa será definida como um *array* de bytes no código.

Sempre que o Comandante adiciona um bombeiro na interface gráfica, o sistema calcula um atraso temporal (offset) com base no Slot que lhe foi atribuído (ex: slot*1500ms). Esse valor é empacotado e enviado num pacote de controlo (indentificado por exemplo com o byte 0xC1). Para garantir que nada falha, o tablet só dá o registo como concluído no ecrã quando recebe do colete um pacote de confirmação (0xC2).

4. Leitura e separação dos dados vitais

Neste código temos uma função que está sempre a ouvir o rádio. Quando deteta um pacote que não é de controlo, desempacota os dados desse pacote (dados vitais). Ao usarmos a formatação <bBHhiih da biblioteca struct, conseguimos pegar nos 16 bytes e convertê-los em variáveis que o código Python consegue ler e processar.

Para otimizar a largura de banda e manter o pacote de rádio o mais leve possível, o firmware do colete agrupa duas informações cruciais no mesmo byte (a variável spo2_header). O primeiro bit à esquerda guarda o estado de Alerta (SOS/Man Down) e os 7 bits restantes guardam a percentagem de oxigénio (SpO2).

Para o Python conseguir separar a informação e ler cada valor individualmente, são feitas as seguintes operações:

- **Para extrair o Alerta (>>7 e & 0x01)**: Como o alerta está na posição mais à esquerda, o comando >>7 empurra os dados 7 "casas" para a direita,

ficando com o alerta no bit mais à direita. A máscara & 0x01 serve como um filtro final que ignora o resto da informação, devolvendo-nos um valor de 1 (Perigo) ou 0 (Normal).

- **Para extrair o SpO2 (& 0x7F):** Neste caso fazemos o inverso. Usamos a máscara 0x7F (01111111 em binário) para apagar o bit de alerta. Desta forma forçamos o bit do alerta a zero, e mantemos os outros 7 bits intactos. O número que sobra é o valor exato da saturação de oxigénio.
- **Ajuste das casas decimais:** Enviar números com vírgulas (*floats*) gasta demasiada memória de rádio. Por isso, o colete multiplica a temperatura por 100 (ex: 36.50°C é enviado na forma 3650) e envia os valores como números inteiros. Após isso vamos pegar nesses números recebidos e dividi-los pelos mesmos multiplicadores para voltarmos a ter as casas decimais corretas no ecrã.

5. Ligação à Interface Gráfica

O próximo passo prático será ligar a função que recebe os dados (*listen_for_incoming_data*) ao ecrã visual do Tablet. Para evitar que a interface fique bloqueada enquanto o rádio está à espera de pacotes, temos que colocar este processo a correr em segundo plano (*threads*). Isto garante que o dashboard responde sempre aos toques e consegue destacar instantaneamente um bombeiro a vermelho mal chegue um sinal de perigo.

6. Como funciona o Código (Communication.py)

6.1 Inicialização e Configuração do Módulo LoRa

```
CS = digitalio.DigitalInOut(board.CE1)
RESET = digitalio.DigitalInOut(board.D25)
spi = busio.SPI(board.SCK, MOSI=board.MOSI, MISO=board.MISO)

try:
    # Inicializar a 868 MHz (Banda ISM Europeia)
    lora = RFM9x(spi, CS, RESET, 868.0)
    # Limitar a potência a 14 dBm para cumprir a lei (ETSI
```

```

EN 300 220)
    lora.tx_power = 14
    print("Rádio LoRa inicializado com sucesso!")
except Exception as e:
    print(f"Erro ao inicializar o LoRa: {e}")

```

- **Mapeamento dos pinos de controlo (CS e RESET):** Através da biblioteca *digitalio*, definimos os pinos que vão gerir o estado da antena. O pino do Chip Select (CE1) é usado para avisar o módulo de que o Raspberry Pi vai lhe enviar comandos, enquanto o pino de *Reset* (D25) garante que o chip é reiniciado e limpa sempre que o nosso código arranca, desta forma prevenimos falhas na memória resultantes de utilizações anteriores.
- **Protocolo de Comunicação SPI:** Ao configurar a variável *spi*, ativamos o sinal de relógio (*Clock*) e os canais de entrada e saída de dados (*MISO* e *MOSI*). Isto abre um canal que garante uma transferência rápida e estável dos nossos pacotes de 16 bytes entre o Raspberry Pi e o rádio.
- **Configuração da Frequência e da Potência de Emissão:** Assim que ativamos o módulo LoRa, colocamo-lo a trabalhar nos 868.0 MHz. Logo na linha seguinte, limitamos a força do sinal a 14 dBm (25 mW). Isto é importante implementar no código para garantir que o projeto funciona dentro da legalidade europeia (ETSI EN 300 220) para esta banda de rádio livre.

6.2 Criação do pacote de controlo

```

def create_packet_config(vest_id, slot):
    Type_packet = 0xC1
    Interval_ms = 10000
    offset = slot * 1500
    epoch = int(time.time())
    packet = struct.pack('<BBHHI6x', vest_id, Type_packet,
        Interval_ms, offset, epoch)
    return packet

```

O prefixo '*<*' obriga o Python a usar a memória em formato *Little-endian*, porque se não o fizermos, o microcontrolador vai receber e ler os números todos trocados. O sufixo '*6x*' no final serve simplesmente para encher o resto

do pacote com zeros (*padding*), obrigando-o a ter os 16 bytes. (Explicação mais detalhada no documento "Communication.py V1")

6.3 Desconstruir a Mensagem do Colete

```
def unpack_sensor_data(decrypted_data):
    #Desempacota os dados biométricos e de GPS de um colete
    try:
        # Formato C++: int8_t, uint8_t, uint16_t, int16_t,
        int32_t, int32_t, int16_t
        v_id, spo2_header, hr_raw, temp_raw, lat_raw, lon_r
        aw, alt_raw = struct.unpack('<bBHhih', decrypted_data)

        return {
            "vestID": v_id,
            "isAlerting": bool((spo2_header >> 7) & 0x01),
# Extrai o bit 7 (Flag)
            "spo2": spo2_header & 0x7F,
# Extrai os 7 bits do SpO2
            "heartRate": hr_raw & 0x01FF,
            "temperature": temp_raw / 100.0,
            "latitude": lat_raw / 100000000.0,
            "longitude": lon_raw / 100000000.0,
            "altitude": alt_raw
        }
    except Exception as e:
        return None
```

Aplicamos a máscara '*<bBHhih*' aos bytes descriptados. O Python corta automaticamente o bloco nas 7 variáveis originais criadas pelo hardware.

- *<bBHhih*:
 - 'b', ID do colete (8 bits com sinal).
 - 'B', Contém o SpO2 e o Alerta (8 bits sem sinal)
 - 'H', Batimento cardíaco (16 bits sem sinal).
 - 'h', Temperatura (16 bits com sinal)

- 'i', Latitude e Longitude (32 bits com sinal cada)
- 'j', Altitude (16 bits com sinal)

6.4 Encriptação e Segurança (AES-128 ECB)

```
def encrypt_aes_ecb(data):
    """Cifra os 16 bytes utilizando AES-128 no modo ECB"""
    cipher = AES.new(AES_KEY, AES.MODE_ECB)
    return cipher.encrypt(data)

def decrypt_aes_ecb(data):
    """Decifra os 16 bytes utilizando AES-128 no modo ECB"""
    cipher = AES.new(AES_KEY, AES.MODE_ECB)
    return cipher.decrypt(data)
```

Neste código é utilizada uma chave simétrica de 16 bytes que é partilhada pelo Master e pelos Slaves(AES_KEY).

O Modo de encriptação implementado é o modo ECB que foi escolhido porque cifra blocos de 16 bytes sem adicionar overhead (como os Vetores de Inicialização exigidos noutros modos). Isto garante que a mensagem por rádio se mantenha compacta e rápida de transmitir, poupando bateria.

6.5 Protocolo de Handshake e Sincronização (ACK)

```
def wait_for_ack(expected_vest_id, timeout=5.0):
    start_time = time.time()
    while (time.time() - start_time) < timeout:
        packet = lora.receive(timeout=0.5)
        if packet is not None and len(packet) == 16:
            decrypted = decrypt_aes_ecb(packet)
            try:
                source_id, p_type, status, battery = struct.unpack('<BBBB12x', decrypted)
                if p_type == 0xC2 and source_id == expected_vest_id:
```

```

        if status == 1:
            print(f"Sucesso: Colete {source_id}
sincronizado! Bateria: {battery}%")
            return True
        except:
            pass
            print(f"Erro: Timeout ao configurar colete {expected_ve
st_id}.")
            return False

```

Após enviar o pacote 0xC1 através da função *send_configuration*, o código entra nesta função e fica à escuta durante 5 segundos. Se receber um pacote, tenta decifrá-lo e verifica se o tipo de pacote (*p_type*) é 0xC2 (que significa Confirmação/ACK). Esta função só retorna *True* (diz à Interface Gráfica que pode "desenhar" o bombeiro no ecrã) se o ID do colete bater certo com o esperado e se o estado for de sucesso (*status==1*). Caso contrário, dá *Timeout*.

6.6 Motor de escuta contínua e Processamento em segundo plano

```

data_queue = queue.Queue()
def radio_listening_thread():

    while True:
        try:
            # Chama a função de escuta
            dados = listen_for_incoming_data()

            if dados:
                # Coloca os novos dados na fila (Queue) par
a a Interface Gráfica ler.
                data_queue.put(dados)
        except Exception as e:
            # Em caso de falha de leitura (ex: ruído no
barramento SPI), faz uma pequena pausa
            time.sleep(1)

```

Para garantir que a Interface Gráfica do Tablet funciona sem interrupções e bloqueios, o processo de escuta da rede LoRa foi colocado a correr em

segundo plano (numa *Background Thread*). Este processo funciona num ciclo contínuo, onde recebe e decifra dados em tempo real sem interferir com o sistema principal. Além disso, no código foi implementado um mecanismo de proteção (try/except) que impede o programa de ir abaixo caso ocorram pequenas falhas de hardware.

6.7 Escuta Contínua e Processamento dos Dados Vitais

```
def listen_for_incoming_data():
    # Função para correr em loop contínuo na Interface Gráfica
    packet = lora.receive(timeout=0.1)

    if packet is not None and len(packet) == 16:
        decrypted_packet = decrypt_aes_ecb(packet)
        packet_type = decrypted_packet[1]

        # Ignora pacotes de controlo (0xC1 e 0xC2)
        if packet_type not in [0xC1, 0xC2]:
            dados = unpack_sensor_data(decrypted_packet)
            if dados:
                print(f"Colete {dados['vestID']} | SpO2: {dados['spo2']}% | BPM: {dados['heartRate']} | Temp: {dados['temperature']}°C")
                # Aqui retorna os dados para a interface (ex: atualizar os labels no ecrã)
                return dados
            return None
    return None
```

A função "listen_for_incoming_data" é o principal ponto de receção de telemetria do Master. Ela é chamada continuamente por uma *thread* em segundo plano. Logo no início, a função faz uma verificação rápida do módulo de rádio, com um tempo de espera (timeout) de 0,1 segundos. Desta forma, o sistema consegue verificar o tráfego aéreo sem ficar bloqueado à espera de dados. Assim que um pacote é intercetado, o código confirma se ele tem exatamente 16 bytes, como exige o algoritmo AES-128 e, se tudo estiver correto, procede à sua descriptação. A lógica central desta função está no seu filtro principal: ao analisar o segundo byte do pacote descriptado, o

sistema ignora propositadamente qualquer mensagem de configuração (0xC1) ou de confirmação (0xC2). A ideia é que esta rotina se concentre apenas na receção dos dados realmente importantes, sobretudo informação vital e de localização dos operacionais. Quando um pacote de telemetria passa por este filtro, o seu conteúdo é desempacotado e convertido num dicionário bem estruturado. No final, a função devolve esta informação já limpa, deixando os dados biométricos prontos para serem injetados e visualizados em tempo real na interface gráfica do utilizador.

6.8 Arranque do Sistema e Ponto de Entrada Principal

```
# ARRANQUE DO PROGRAMA
if __name__ == "__main__":

    # 1. Pára a execução se o hardware do rádio falhar
    if 'lora' not in globals():
        print("O sistema não pode arrancar sem o módulo LoRa.")
        exit(1)

    # 2. Iniciar a Thread de escuta de dados
    rx_thread = threading.Thread(target=radio_listening_thread, daemon=True)
    rx_thread.start()

    print("\n Serviço de comunicações a correr...")
    print("Ctrl+C para encerrar.\n")

    # 3. Manter o script principal a correr
    try:
        while True:
            # Aqui é onde vamos encaixar a Interface Gráfica
            time.sleep(1)
```

```
except KeyboardInterrupt:  
    print("\n Encerramento feito pelo utilizador.")
```

Este é o bloco final do código e funciona como o “motor de arranque” do sistema. É ele que orchestra tudo o que acontece a partir do momento em que executamos o ficheiro.

Logo no início é feito um teste de segurança: o sistema verifica se o módulo de rádio LoRa arrancou corretamente e se está disponível na memória. Se houver algum problema físico, como por exemplo um cabo SPI mal ligado no Raspberry Pi, o programa é imediatamente interrompido. Isto impede que o software tente comunicar com um rádio que não existe, o que poderia levar a bloqueios na aplicação.

Depois de confirmado que o módulo de rádio está operacional, é iniciada a escuta da rede, arrancando a thread de comunicações em segundo plano. De seguida, o script entra num ciclo contínuo de espera (while True), que, nesta fase, ainda está incompleto, pois é aqui que vamos ligar a interface gráfica.

Por último, foi incluída uma forma segura de desligar o sistema “KeyboardInterrupt”. Se for necessário parar o programa manualmente, ele termina tudo de forma limpa.